



Integración de Memoria y Razonamiento en tu Estructura `.ai/`

Excelente material. Has compartido una arquitectura híbrida muy completa (tu solución + mejoras de Engram). Ahora te muestro cómo integrarlo en tu estructura `.ai/` de forma práctica y estandarizada.

1. Estructura Final Recomendada

text

```
mi-proyecto/
├── .ai/
│   ├── system.md           # Prompt sistemático (oficial OpenCode)
│   ├── agents.md           # Agentes especialistas
│   ├── rules.md            # Reglas globales
│   ├── context.md          # Contexto técnico
│   └── memory.md           # ← NUEVO: Instrucciones de memoria para el agente
├── .ai_memory/             # ← NUEVO: Base de datos de memoria (ChromaDB)
│   ├── chroma.sqlite3
│   └── export.json         # Para Git sync
├── ai_architect/           # ← NUEVO: Motor de memoria (código Python)
│   ├── core/
│   └── __init__.py
```

```

├── router.py           # Multi-proveedor con fallback
├── memory.py          # Memoria estructurada + conflictos
├── pipelines.py       # Cadenas LangChain (CoT + citas)
├── mcp_server.py      # Servidor MCP para OpenCode/Cursor
├── ui/
│   ├── __init__.py
│   ├── cli.py         # CLI para operaciones
│   └── gradio_app.py  # UI web opcional
├── config/
│   ├── __init__.py
│   ├── settings.py    # Configuración centralizada
│   └── prompts.py     # Prompts reutilizables
├── requirements.txt
└── main.py            # Punto de entrada unificado

.vscode/
└── settings.json

.env                # API keys (GROQ, OPENROUTER, GOOGLE, etc.)
.gitignore
opencode.json       # Configuración de OpenCode + MCP
src/
└── ...código fuente...

```

2. Archivos Clave a Crear/Modificar

A. `.ai/memory.md` (Nuevo)

Este archivo le dice al agente CÓMO usar la memoria:

Markdown

```

# 🧠 Memoria del Proyecto

## Sistema de Memoria Activo
Este proyecto utiliza un **sistema de memoria persistente** con las siguientes capacidades:

### Tipos de Memoria
| Tipo | Descripción | Ejemplo |
|-----|-----|-----|
| `episodic` | Experiencias y conversaciones | "Bug resuelto en auth.py" |
| `decision` | Decisiones técnicas | "Usar ChromaDB sobre SQLite" |
| `architecture` | Patrones y estructura | "Arquitectura hexagonal" |
| `bugfix` | Soluciones a errores | "Fix: null pointer en usuario" |
| `tutorial` | Aprendizajes documentados | "Cómo configurar MCP" |
| `reference` | Documentación indexada | "API endpoints v2" |

### Herramientas MCP Disponibles
El agente puede usar estas herramientas **vía MCP**:

```

/architect_save → Guardar memoria con detección de conflictos
/architect_search → Búsqueda semántica en memoria
/architect_generate → Generar respuesta con citas y anti-alucinación
/architect_verify → Verificar afirmaciones con evidencia
/architect_status → Estado del sistema (proveedores, memoria, salud)
/architect_export → Exportar memorias para Git sync

text

```
### Reglas de Memoria
1. **SIEMPRE** guarda decisiones importantes con `/architect_save`
2. **ANTES** de responder, busca en memoria con `/architect_search`
3. **CITA** la fuente de cada afirmación factual [1], [2]
4. **VERIFICA** con `/architect_verify` si hay números o claims críticos
5. **EXPORTA** periódicamente con `/architect_export` para Git sync

### Ubicación de la Memoria
- Base de datos: `.ai_memory/`
- Exportaciones: `.ai_memory/export.json`
- **NO** commitear `chroma.sqlite3` (ver `.gitignore`)

- **Sí** commitear `export.json` (opcional, para sync manual)
```

B. `opencode.json` (Modificar)

Configura el servidor MCP para que OpenCode/Cursor usen tu memoria:

JSON

```
{
  "mcpServers": {
    "ai-architect": {
      "command": "python",
      "args": ["ai_architect/main.py", "--ui", "mcp"],
      "env": {
        "PROJECT_NAME": "mi-proyecto"
      }
    }
  },
  "agent": {
    "systemPrompt": ".ai/system.md",
    "context": [
      ".ai/context.md",
      ".ai/rules.md",
      ".ai/agents.md",
      ".ai/memory.md"
    ]
  }
}
```

C. .gitignore (Actualizar)

gitignore

```
# AI Architect Hybrid
.ai_memory/chroma.sqlite3
.ai_memory/__pycache__/.
.ai_memory/*.pyc

# Secrets
.env
!.env.example

# Cache
.gradio/
.pytest_cache/

# Exportaciones (opcional: commitear si quieres sync manual)
# .ai_memory/export.json
```

D. .env (Actualizar)

Bash

```
# Proveedores (obtenlas gratis):
GROQ_API_KEY=gsk_...
OPENROUTER_API_KEY=sk-or-...
GOOGLE_API_KEY=Alza...

# Opcional: proveedor preferido
PRIMARY_PROVIDER=groq # groq | openrouter | google

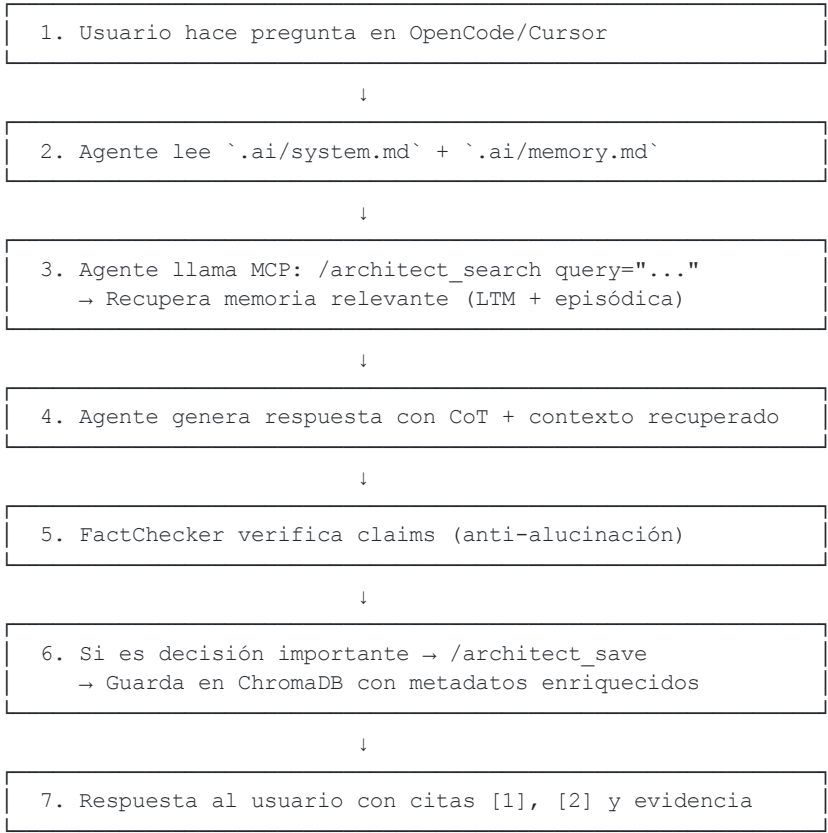
# Memoria
CHROMA_PERSIST_DIR=./.ai_memory
EMBEDDING_MODEL=all-MiniLM-L6-v2

# Guardrails
FACT_CHECK_MIN_CONFIDENCE=0.65
REQUIRE_CITATION_WHEN_CLAIM_HAS_NUMBERS=true
ENABLE_WEB_SEARCH=true
```

3. Flujo de Trabajo Integrado

Cuando el Agente Trabaja:

text



4. Comandos Prácticos (CLI)

Una vez instalado (`pip install -r ai_architect/requirements.txt`):

Comando	Función
<code>python ai_architect/main.py --ui gradio</code>	Inicia UI web (http://localhost:7860)

```
python ai_architect/main.py --ui cli status
```

Ver estado del sistema

```
python ai_architect/main.py --ui cli export  
--preview
```

Exportar memorias (vista
previa)

```
python ai_architect/main.py --ui cli import  
../otro-proyecto/.ai_memory/export.json
```

Importar de otro proyecto

```
python ai_architect/main.py --ui cli search  
"implementar RAG" --type architecture --limit 3
```

Búsqueda rápida

```
python ai_architect/main.py --ui mcp
```

Iniciar servidor MCP (para
OpenCode/Cursor)

5. Mejoras Clave que Estás Obteniendo

Feature

Beneficio

Memoria estructurada	Metadatos enriquecidos (title, type, what, why, learned, tags)
----------------------	--

Detección de conflictos	LLM-judge gratuito evita contradicciones
-------------------------	--

Multi-proveedor	Fallback automático (Groq → OpenRouter → Google)
-----------------	--

Anti-alucinación	Citas + verificación antes de responder
------------------	---

Git sync ready	Export/import JSON portable entre proyectos
----------------	---

MCP nativo	Compatible con Claude Code, Cursor, OpenCode sin código adicional
------------	---

Cache inteligente	Reduce llamadas a LLM y mejora velocidad
-------------------	--

Health checks	Detecta proveedores caídos y hace fallback automático
---------------	---



6. Pasos de Implementación (Checklist)

Markdown

- [] 1. Crear carpeta ``ai_architect/`` con el código Python compartido
- [] 2. Crear ``ai/memory.md`` con las instrucciones para el agente
- [] 3. Actualizar ``opencode.json`` con configuración MCP
- [] 4. Actualizar ``gitignore`` para excluir ``chroma.sqlite3``
- [] 5. Actualizar ``env`` con las API keys de proveedores
- [] 6. Instalar dependencias: ``pip install -r ai_architect/requirements.txt``
- [] 7. Probar CLI: ``python ai_architect/main.py --ui cli status``
- [] 8. Iniciar servidor MCP: ``python ai_architect/main.py --ui mcp``
- [] 9. Conectar OpenCode/Cursor (reiniciar editor)
- [] 10. Probar herramienta: ``/architect_search query="mi primera consulta"``
- [] 11. (Opcional) Iniciar UI web: ``python ai_architect/main.py --ui gradio``

7. Ejemplo de Uso en el Chat del Agente

text

Usuario: ¿Cómo manejamos los rate limits en este proyecto?

Agente (automáticamente):

1. `/architect_search query="rate limits"` → Encuentra 3 memorias relevantes
2. Genera respuesta con CoT + contexto recuperado
3. `/architect_verify claims con números` → Verifica
4. Responde con citas [1], [2], [3]
5. `/architect_save` (si es decisión nueva) → Guarda en memoria

Respuesta:

"Según nuestra documentación [1] y la decisión arquitectónica [2], implementamos un router con fallback automático. Los rate limits son: Groq (30 RPM), OpenRouter (20 RPM), Google (15 RPM) [3].

Verificación: ✓ Todos los números tienen fuente en memoria."



Resumen

Elemento	Ubicación	Función
<code>system.md</code>	<code>.ai/</code>	Prompt sistemático (oficial OpenCode)
<code>memory.md</code>	<code>.ai/</code>	Instrucciones de memoria para el agente
<code>ai_architect/</code>	Raíz	Motor de memoria (Python + ChromaDB + MCP)
<code>.ai_memory/</code>	Raíz	Base de datos persistente (no commitear SQLite)
<code>opencode.json</code>	Raíz	Configuración MCP para OpenCode/Cursor
<code>.env</code>	Raíz	API keys de proveedores (Groq, OpenRouter, Google)

Con esta integración, tu agente acumula experiencia con cada proyecto, no alucina (verifica con evidencia), aprende de decisiones pasadas y puede compartir memoria entre proyectos vía Git sync. 🚀